

# **Numerical Methods in Soft Matter Exercises**

Guglielmo Bordin

University of Padua, 2024



---

## Contents

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Sampling via hit-and-miss and inversion</b>                       | <b>1</b>  |
| 1.1      | Sampling in $d$ -dimensional domains by hit-and-miss . . . . .       | 1         |
| 1.2      | Sampling from a distribution with the inversion method . . . . .     | 1         |
| 1.3      | Another sampling with the inversion method . . . . .                 | 1         |
| 1.4      | Additional exercises on the inversion method . . . . .               | 3         |
| <b>2</b> | <b>Sampling via transformation of coordinates</b>                    | <b>7</b>  |
| 2.1      | Sampling uniformly within a unit radius disk . . . . .               | 7         |
| 2.2      | Box–Muller transform . . . . .                                       | 7         |
| 2.3      | Rejection method . . . . .                                           | 8         |
| <b>3</b> | <b>Importance sampling</b>                                           | <b>11</b> |
| 3.1      | Comparing importance sampling to the ‘crude’ method . . . . .        | 11        |
| 3.2      | Optimizing importance sampling for cosine integration . . . . .      | 12        |
| 3.3      | Analysing the variance with or without importance sampling . . . . . | 14        |
| <b>4</b> | <b>Markov chains</b>                                                 | <b>17</b> |
| 4.1      | Rewriting the recurrence relation . . . . .                          | 17        |
| 4.2      | Drawing digraphs . . . . .                                           | 17        |
| 4.3      | Proving the irreducibility of a chain . . . . .                      | 18        |
| 4.4      | Proving the regularity of a chain . . . . .                          | 19        |
| 4.5      | Proving the stationarity of a chain . . . . .                        | 20        |
| <b>5</b> | <b>Ising model with the Metropolis algorithm</b>                     | <b>21</b> |
| 5.1      | Simulations at different temperatures . . . . .                      | 21        |
| <b>6</b> | <b>Advanced simulations of the Ising model</b>                       | <b>23</b> |
| 6.1      | Wolff cluster algorithm . . . . .                                    | 23        |
| 6.2      | Multiple Markov chains . . . . .                                     | 23        |
| <b>7</b> | <b>Continuous-time Markov processes</b>                              | <b>27</b> |
| 7.1      | Lotka–Volterra model . . . . .                                       | 27        |
| 7.2      | Brusselator model . . . . .                                          | 27        |
| <b>8</b> | <b>Off-lattice Monte Carlo simulations</b>                           | <b>35</b> |
| 8.1      | Reduced units . . . . .                                              | 35        |
| 8.2      | Off-lattice Monte Carlo program prototype . . . . .                  | 35        |
| 8.3      | Off-lattice Monte Carlo of hard spheres . . . . .                    | 36        |



## Sampling via hit-and-miss and inversion

### 1.1 Sampling in $d$ -dimensional domains by hit-and-miss

*Source codes*      `src/011a_rect_hit_miss.c, src/011b_disk_hit_miss.c`

*Analysis script*   `src/011_hit_miss.R`

*Executables*      `exe/011a_rect_hit_miss, exe/011b_disk_hit_miss`

The error as a function of the number of throws is shown in fig. 1.1.1. In the disk case, it goes under 1 % with as little as 10 000 ‘throws’.

### 1.2 Sampling from a distribution with the inversion method

*Source code*      `src/012_inversion_power34.c`

*Analysis script*   `src/012-4_inversion_method.R`

*Executable*      `exe/012_inversion_power34`

The idea is to sample from the probability distribution  $\rho_n(x) = cx^n$  in  $[0, 1]$ . First, using the normalization condition we can find out what  $c$  should be:

$$1 = \int_0^1 cx^n dx = \frac{c}{n+1} \implies c = n+1. \quad (1.2.1)$$

Then, we find the expression of the associated cumulative density function:

$$F_n(x) = (n+1) \int_0^x y^n dy = x^{n+1}, \quad (1.2.2)$$

and invert it:

$$u = x^{n+1} \implies x = u^{1/(n+1)}. \quad (1.2.3)$$

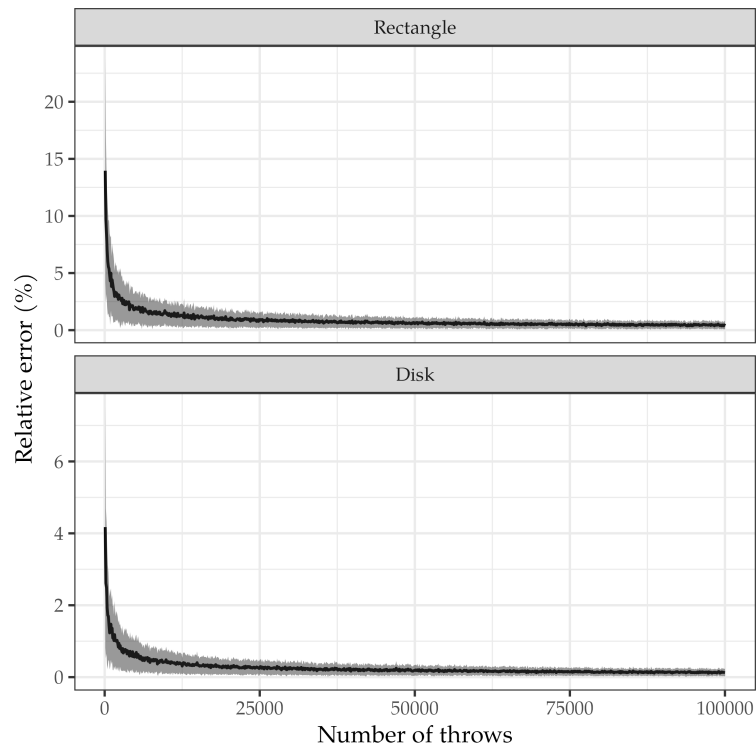
So, we just have to sample a random floating point number between 0 and 1 and raise it to the power of  $1/(n+1)$  to get  $x \sim (n+1)x^n$ . Results from a sampling of 50 000 iterations for  $n = 3, 4$  are shown in fig. 1.2.1.

### 1.3 Another sampling with the inversion method

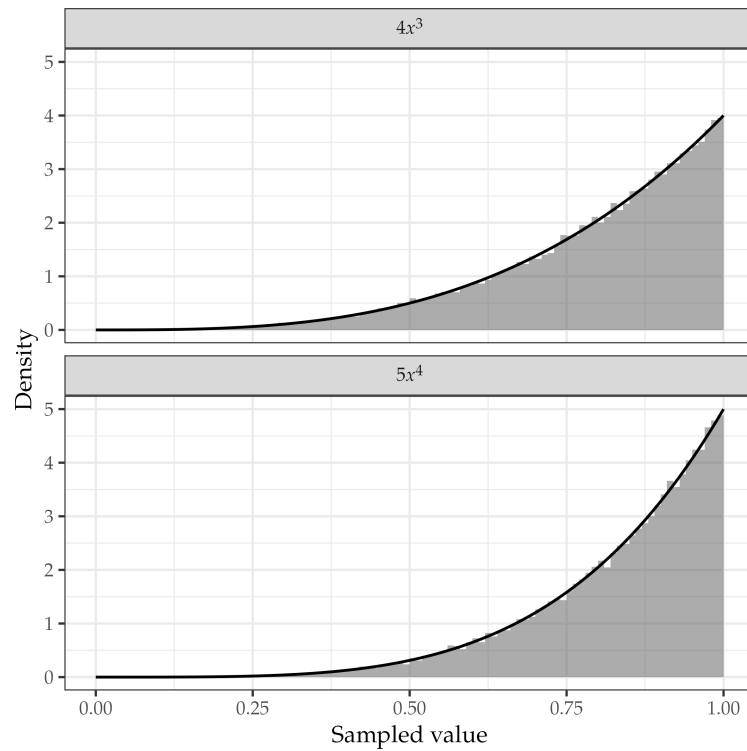
*Source code*      `src/013_inversion_power2.c`

*Analysis script*   `src/012-4_inversion_method.R`

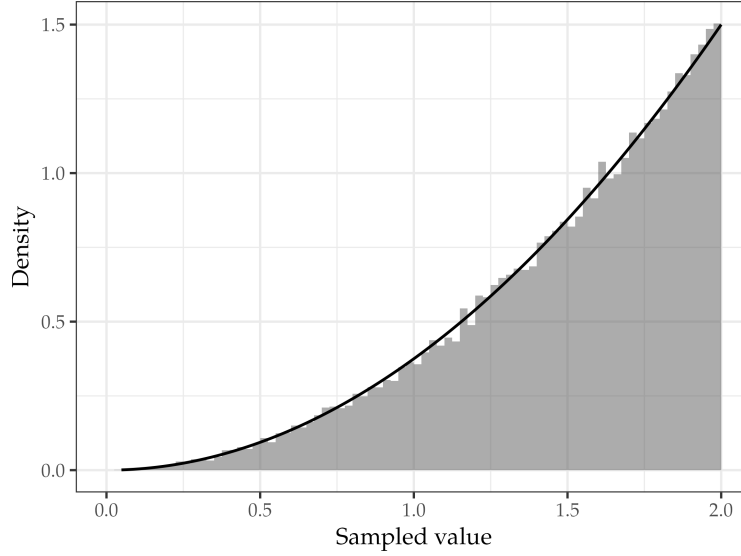
*Executable*      `exe/013_inversion_power2`



**Figure 1.1.1:** percentage error as a function of the number of ‘throws’ for exercise 1.1. The black line represents the average over 100 realizations, with the standard deviation shaded in grey.



**Figure 1.2.1:** histogram of 50 000 samples from  $\rho_n(x) = cx^n$ ,  $x \in [0, 1]$ , for  $n = 3$  (above) and  $n = 4$  (below), obtained with the inversion method. The theoretical distribution is overlaid with a solid line.



**Figure 1.3.1:** histogram of 50 000 samples from  $\rho(x) = 3x^2/8$ ,  $x \in [0, 2]$ , obtained with the inversion method. The theoretical distribution is overlaid with a solid line.

I slightly modified the code from the previous exercise, since  $c$  is different now. The distribution is  $\rho(x) = cx^2$  with  $x \in [0, 2]$ , so:

$$1 = \int_0^2 cx^2 dx = \frac{8}{3}c \implies c = \frac{3}{8}. \quad (1.3.1)$$

The cumulative is then

$$F(x) = \frac{3}{8} \int_0^x y^2 dy = \frac{x^3}{8} \implies x = 2u^{1/3}. \quad (1.3.2)$$

Once again, you can see the comparison between a 50 000-points histogram and the theoretical curve in fig. 1.3.1.

#### 1.4 Additional exercises on the inversion method

*Source codes*      `src/014a_inversion_exp.c`, `src/014b_inversion_exp2.c`,  
                          `src/014c_inversion_powerlaw.c`

*Analysis script*   `src/012-4_inversion_method.R`

*Executables*       `exe/014_inversion_exp`, `exe/014b_inversion_exp2`,  
                          `exe/014c_inversion_powerlaw`

The first exercise here asked to sample from  $\rho_1(x) = \mu e^{-\mu x}$ , with  $x \in [0, \infty)$ . The cumulative and the sampling rule are then

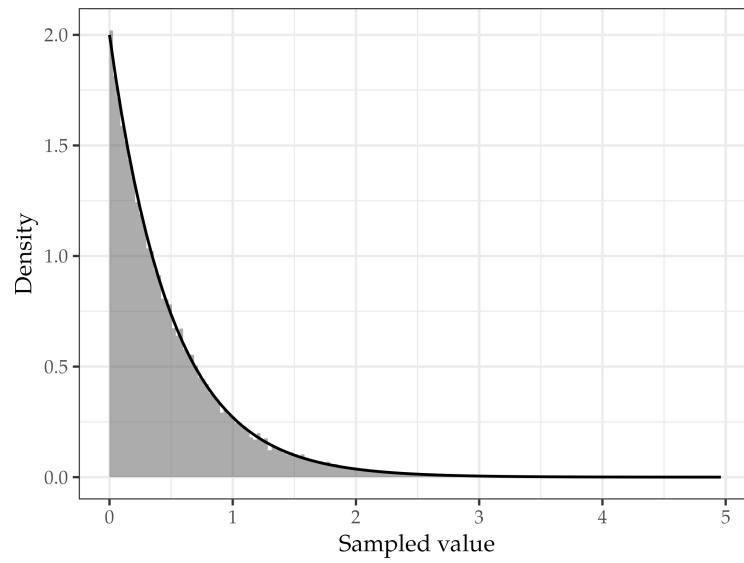
$$F_1(x) = \mu \int_0^x e^{-\mu y} dy = \mu(1 - e^{-\mu x}) \implies x = -\log(1 - u)/\mu. \quad (1.4.1)$$

I ran 50 000 samples as before, choosing  $\mu = 2$ ; see fig. 1.4.1.

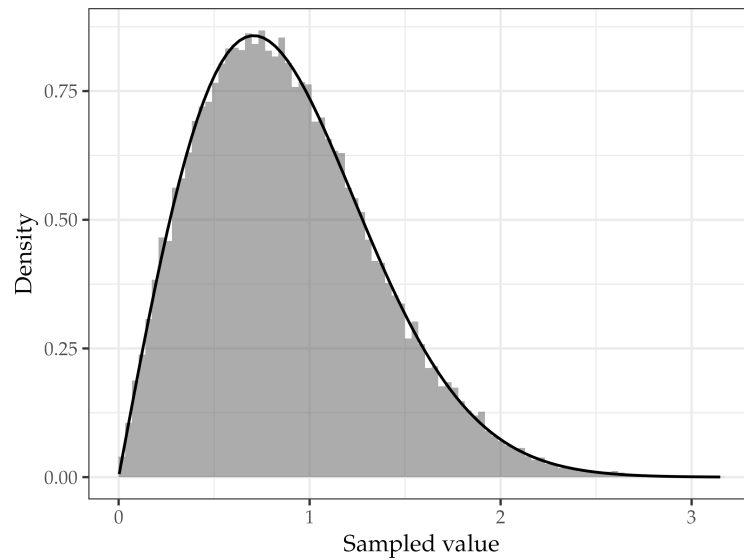
The second distribution was  $\rho_2(x) = 2xe^{-x^2}$ ,  $x \in [0, \infty)$ . Cumulative and sampling rule:

$$F_2(x) = 2 \int_0^x e^{-y^2} dy = 1 - e^{-x^2} \implies x = \sqrt{-\log(1 - u)}. \quad (1.4.2)$$

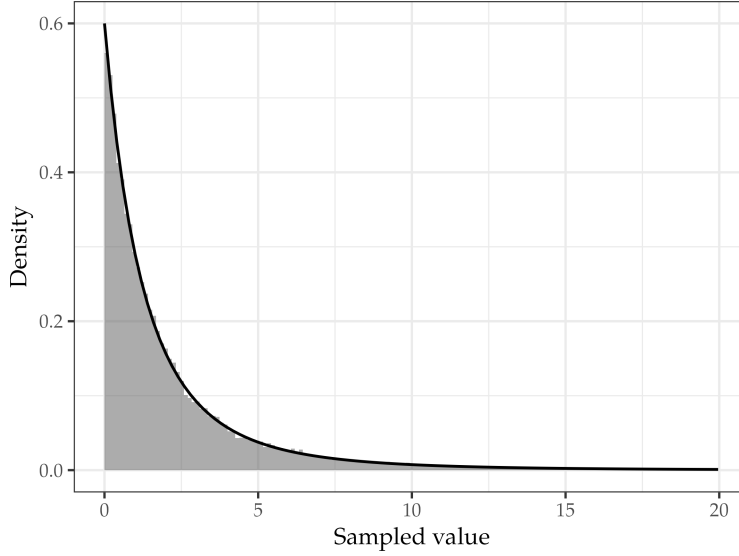
A histogram of 50 000 samples is shown in fig. 1.4.2.



**Figure 1.4.1:** histogram of 50 000 samples from  $\rho(x) = 2e^{-2x}$ ,  $x \in [0, \infty)$ , obtained with the inversion method. The theoretical distribution is overlaid with a solid line.



**Figure 1.4.2:** histogram of 50 000 samples from  $\rho(x) = 2xe^{-x^2}$ ,  $x \in [0, \infty)$ , obtained with the inversion method. The theoretical distribution is overlaid with a solid line.



**Figure 1.4.3:** histogram of 50 000 samples from  $\rho(x) = 375/(5+x)^4$ ,  $x \in [0, \infty)$ , obtained with the inversion method. The theoretical distribution is overlaid with a solid line.

The last distribution was  $\rho_3(x) = c(a+bx)^{-n}$ , with  $x \in [0, \infty)$  and  $n > 1$ . The normalization constant is given by

$$1 = \int_0^\infty c(a+bx)^{-n} dx = \frac{a^{1-n}c}{b(n-1)} \implies c = b(n-1)a^{n-1}. \quad (1.4.3)$$

By integrating we obtain

$$F_3(x) = c \int_0^x (a+by)^{-n} dy = \frac{c}{b(1-n)} [(a+bx)^{1-n} - a^{1-n}] = 1 - a^{n-1}(a+bx)^{1-n}, \quad (1.4.4)$$

from which we can sample with

$$u = 1 - a^{n-1}(a+bx)^{1-n} \implies x = \frac{a}{b} [(1-u)^{1/(1-n)} - 1]. \quad (1.4.5)$$

I chose  $a = 1/2$ ,  $b = 1/10$  and  $n = 4$  – resulting in  $\rho_3(x) = 375/(5+x)^4$  – for a run of 50 000 samples, shown in fig. 1.4.3.



## Sampling via transformation of coordinates

### 2.1 Sampling uniformly within a unit radius disk

*Source code*      `src/021_disk_sampling.c`

*Analysis script*   `src/021_disk_sampling.R`

*Executable*        `exe/021_disk_sampling`

If we want to sample points uniformly distributed over the unit disk, we cannot simply generate  $r \sim \mathcal{U}_{[0,1]}$  and  $\vartheta \sim \mathcal{U}_{[0,2\pi]}$ . The result of doing that is shown in the left graph of fig. 2.1.1. While the angular distribution poses no problem, there is an undesired higher density of points close to the centre of the disk.

The reason for this becomes clear when you consider the number of points falling inside a ring of given thickness  $\delta r$ . Consider a ring with inner radius  $r_1$  and outer radius  $r_1 + \delta r$ , with  $\delta r \ll r_1$ , and another with radii  $r_2, r_2 + \delta r$ . Despite the ratio of their areas being

$$\frac{\pi[(r_1 + \delta r)^2 - r_1^2]}{\pi[(r_2 + \delta r)^2 - r_2^2]} = \frac{2r_1\delta r + \mathcal{O}(\delta r^2)}{2r_2\delta r + \mathcal{O}(\delta r^2)} = \frac{r_1}{r_2} + \mathcal{O}(\delta r), \quad (2.1.1)$$

the fraction of points within each ring is in both cases proportional to  $\delta r$ , if  $r$  is sampled uniformly. Thus, the number of points falling at a distance  $r$  should be proportional to  $r$ :

$$\rho_r(r) = 2r, \quad (2.1.2)$$

with the 2 in front to ensure normalization over  $[0, 1]$ . By computing the cumulative and inverting we get

$$F_r(r) = 2 \int_0^r x \, dx = r^2 \implies u = r^2 \implies r = \sqrt{u}. \quad (2.1.3)$$

The graph on the right in fig. 2.1.1 shows how the sampling becomes visibly uniform with this adjustment.

### 2.2 Box–Muller transform

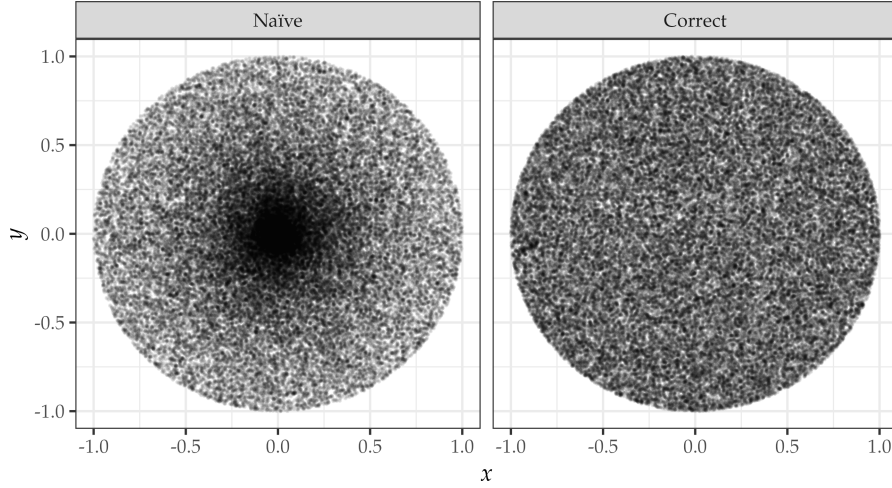
*Source code*      `src/022_box_muller.c`

*Analysis script*   `src/022_box_muller.R`

*Executable*        `exe/022_box_muller`

I started from the factorization of  $\rho_{x,y}(x, y)$ :

$$\rho_{x,y}(x, y) = \frac{1}{2\pi} \exp\left(-\frac{x^2 + y^2}{2}\right) = \rho_x(x)\rho_y(y), \quad \text{with } \rho_x(x) = \frac{e^{-x^2/2}}{\sqrt{2\pi}}. \quad (2.2.1)$$



**Figure 2.1.1:** visualization of 50 000 samples on the unit disk, with the ‘naïve’ method of uniform sampling both on  $r$  and  $\vartheta$  (left), and with the correct method described in the text (right).

If we switch to polar coordinates this becomes, remembering the factor  $r$  coming from the Jacobian,

$$\rho_{r,\vartheta}(r, \vartheta) = \frac{r}{2\pi} e^{-r^2/2}, \quad (2.2.2)$$

with  $\rho_r(r) = r e^{-r^2/2}$  and  $\rho_\vartheta(\vartheta) = 1/2\pi$ .

Then, we can marginalize over  $\vartheta$  to get  $\rho_r(r)$ :

$$\rho_r(r) = \frac{1}{2\pi} \int_0^{2\pi} r e^{-r^2/2} d\vartheta = r e^{-r^2/2}. \quad (2.2.3)$$

As usual, we compute the cumulative and invert it to generate a random  $r \sim \rho_r(r)$ :

$$F_r(r) = \int_0^r x e^{-x^2/2} dx = 1 - e^{-r^2/2} \implies e^{-r^2/2} = 1 - u \implies r = \sqrt{-2 \log(1 - u)}. \quad (2.2.4)$$

Then, to get the angle  $\vartheta$  we recover the *conditional* distribution  $\rho_{\vartheta|r}(\vartheta | r)$ :

$$\rho_{\vartheta|r}(\vartheta | r) = \frac{\rho_{r,\vartheta}(r, \vartheta)}{\rho_r(r)} = \frac{1}{2\pi}. \quad (2.2.5)$$

Thus, for  $\vartheta$  we have simply to sample from  $\mathcal{U}_{[0,2\pi]}$ .

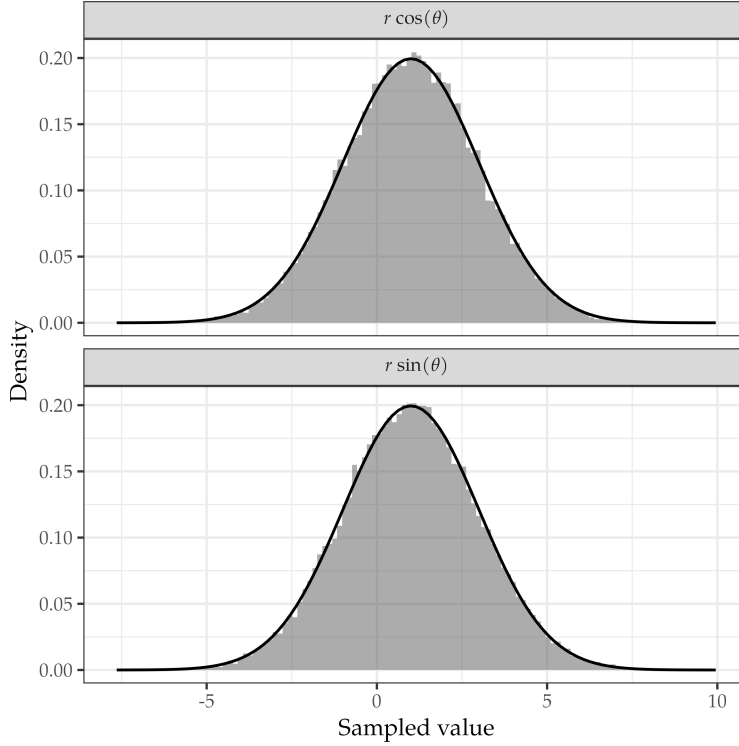
The idea then is to sample two numbers  $u_1, u_2$  from  $\mathcal{U}_{[0,1]}$  at each iteration; at that point we can calculate

$$x = \sqrt{-2 \log u_1} \cos(2\pi u_2), \quad y = \sqrt{-2 \log u_1} \sin(2\pi u_2), \quad (2.2.6)$$

to get two numbers  $x, y$  distributed according to a standard Gaussian  $\mathcal{N}(0, 1)$ . We can also get a  $z \sim \mathcal{N}(\mu, \sigma^2)$  afterwards by multiplying  $x$  or  $y$  by  $\sigma$  and summing the mean:  $z = \mu + \sigma x$ . I ran a simulation with 100 000 samples choosing  $\mu = 1$  and  $\sigma = 2$ , and you can see the results in fig. 2.2.1.

### 2.3 Rejection method

|                        |                                                                 |
|------------------------|-----------------------------------------------------------------|
| <i>Source codes</i>    | src/023a_rejection_sampling.c,<br>src/023b_rejection_accratio.c |
| <i>Analysis script</i> | src/023_rejection_method.R                                      |
| <i>Executables</i>     | exe/023a_rejection_sampling,<br>exe/023b_rejection_accratio     |



**Figure 2.2.1:** histograms of 100 000 samples – half in the upper panel, half in the lower one – from a  $\mathcal{N}(1, 4)$  distribution, obtained with the Box–Muller method. The theoretical distribution is overlaid with a solid line.

Here I implemented the sampling of random numbers from the distribution  $f(x) = 2e^{-x^2}/\sqrt{\pi}$  over  $[0, \infty)$ . We start from a function  $g(x)$  that could serve as an upper limit to  $f(x)$ ,

$$g(x) = \begin{cases} A & \text{for } 0 \leq x \leq p, \\ \frac{A}{p} x e^{p^2 - x^2} & \text{for } x > p. \end{cases} \quad (2.3.1)$$

First we normalize it to turn it into a proper probability distribution:

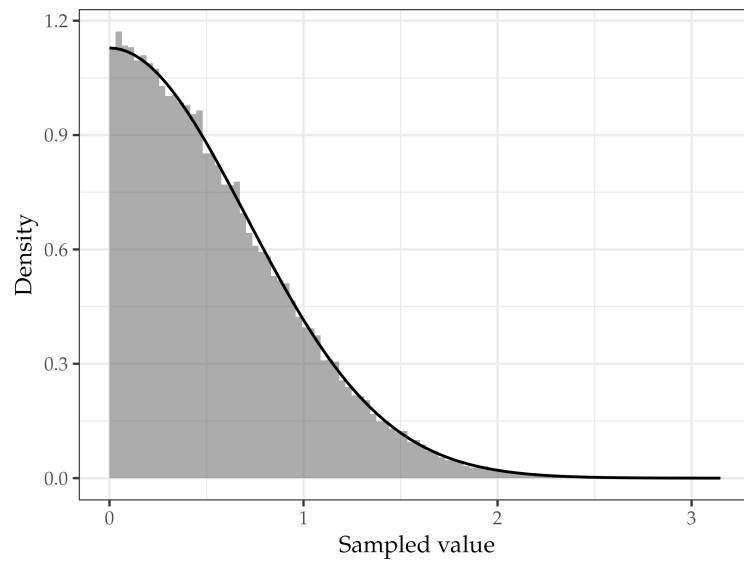
$$1 = \int_0^\infty g(x) dx = Ap + \frac{A}{p} e^{p^2} \int_p^\infty x e^{-x^2} dx = A \left( p + \frac{1}{2p} \right) \implies A = \frac{2p}{1 + 2p^2}. \quad (2.3.2)$$

Then we need a constant  $c$  such that  $c g(x) \geq f(x)$  everywhere. The maximum of  $f(x)$  is in  $x = 0$ , where  $f(0) = 2/\sqrt{\pi}$ , so we set  $cA = 2/\sqrt{\pi}$ . Another thing to consider is that  $g(x)$  can have a maximum larger than  $A$  in  $[p, \infty)$ :

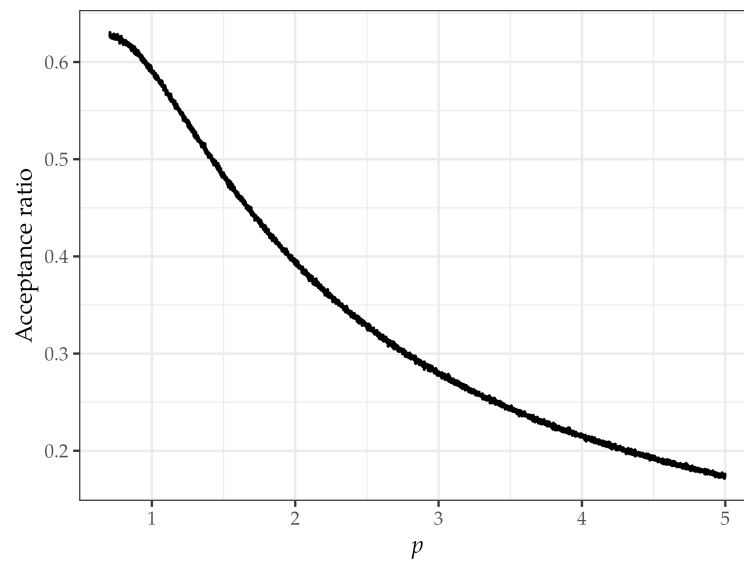
$$g'(x) = \begin{cases} 0 & \text{for } 0 \leq x \leq p, \\ \frac{A}{p} (1 - 2x^2) e^{p^2 - x^2} & \text{for } x > p. \end{cases} \quad (2.3.3)$$

Thus in the region  $[p, \infty)$  the derivative is zero in  $x = 1/\sqrt{2}$ . To avoid this we have to choose  $p \geq 1/\sqrt{2} \approx 0.707$ . An example of sampling with  $p = 1/\sqrt{2}$  is given by the histogram in fig. 2.3.1. A plot of the acceptance ratio as a function of  $p$ , starting from the minimum value  $1/\sqrt{2}$ , is shown in fig. 2.3.2; obviously the acceptance ratio decreases as  $p$  increases, as the gap between  $c g(x)$  and  $f(x)$  grows wider:

$$\begin{aligned} \int_0^\infty (c g(x) - f(x)) dx &= \frac{2}{\sqrt{\pi}} \int_0^p dx + \frac{2}{p\sqrt{\pi}} \int_p^\infty x e^{p^2 - x^2} dx - 1 \\ &= \frac{2}{\sqrt{\pi}} \left( p + \frac{1}{2p} \right) - 1 \xrightarrow{p \rightarrow \infty} \infty. \end{aligned} \quad (2.3.4)$$



**Figure 2.3.1:** histogram of 50 000 samples from the distribution  $f(x) = 2e^{-x^2}/\sqrt{\pi}$  in  $[0, \infty)$  obtained with the rejection method. The theoretical distribution is overlaid with a solid line.



**Figure 2.3.2:** acceptance ratio over 100 000 samples as a function of  $p$  in exercise 2.3.

## Importance sampling

### 3.1 Comparing importance sampling to the ‘crude’ method

*Source code*      `src/031_crude_vs_importance.c`

*Analysis script*   `src/031_crude_vs_importance.R`

*Executable*      `exe/031_crude_vs_importance`

The goal was to integrate the function  $f(x) = g(x)e^{-x^2}$  on  $[0, \infty)$ , with  $g(x)$  being a slowly varying function. I chose  $g(x) = x \cos x^2$ , so the exact value of the integral is

$$\int_0^\infty f(x) dx = \int_0^\infty x \cos(x^2) e^{-x^2} dx = \dots = \frac{1}{4}. \quad (3.1.1)$$

Following the suggestion, we multiply and divide by a weighting function defined as

$$w(x) = \frac{2}{\sqrt{\pi}} e^{-x^2}, \quad (3.1.2)$$

and so we can write

$$\int_0^\infty g(x) e^{-x^2} dx = \frac{\sqrt{\pi}}{2} \int_0^\infty g(x) w(x) dx = \frac{\sqrt{\pi}}{2} \int_0^\infty g(x) dw(x). \quad (3.1.3)$$

Therefore, we can sample numbers  $x_i$  from  $w(x)$  and estimate the integral with

$$\int_0^\infty g(x) e^{-x^2} dx \approx \frac{\sqrt{\pi}}{2N} \sum_{i=1}^N g(x_i). \quad (3.1.4)$$

To sample from  $w(x)$ , which is a half-normal distribution with variance  $1/2$ , we can use a modified Box-Muller. The idea is to restrict the number generation to the  $x > 0$  half-plane. First, since in this case the variance is not unitary, we can recompute the cumulative. The marginal distribution over  $r$  needs a factor 2 in front for normalization, and we obtain

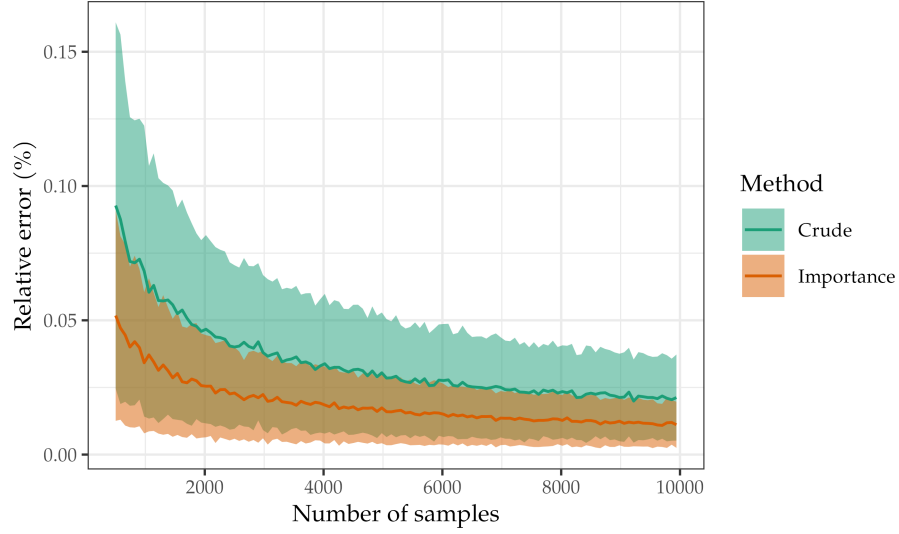
$$\rho_r(r) = 2r e^{-r^2} \implies F_r(r) = 1 - e^{-r^2} \implies r = \sqrt{-\log(1 - u_1)}, \quad \text{with } u_1 \sim \mathcal{U}_{[0,1]}. \quad (3.1.5)$$

For  $\vartheta$  we sample uniformly over  $[-\pi/2, +\pi/2]$  in order to stay in the  $x > 0$  half-plane:

$$\vartheta = -\frac{\pi}{2} + \pi u_2, \quad \text{with } u_2 \sim \mathcal{U}_{[0,1]}. \quad (3.1.6)$$

Once we have  $r$  and  $\vartheta$  we generate  $x, y$  distributed according to  $w(x)$  in this way:

$$x = r \cos \vartheta, \quad y = r |\sin \vartheta|. \quad (3.1.7)$$



**Figure 3.1.1:** comparison of the relative error between a ‘crude’ Monte Carlo integration and importance sampling as a function of the number of samples. The solid lines represent the average over 500 realizations, with the standard deviation shaded in the same colour.

In contrast to the importance sampling technique, inside the same C file I also implemented the ‘crude’ Monte Carlo integration, that is a simple average of a set of  $f(x_i)$  with  $x_i$  sampled uniformly over an interval from 0 to a given  $x_{\max}$ :

$$\int_0^\infty g(x)e^{-x^2} dx \approx \frac{1}{N} \sum_{i=1}^N g(x_i)e^{-x_i^2}, \quad \text{with } x_i \sim \mathcal{U}_{[0, x_{\max}]}. \quad (3.1.8)$$

You can see the comparison between the two methods in terms of relative error in fig. 3.1.1. The importance sampling is definitely more efficient, although the crude method has a higher variance which can lead to a lower error in lucky runs. Also, as the number of samples grows the difference between the two methods shrinks, but this is rather unsurprising.

### 3.2 Optimizing importance sampling for cosine integration

Source code      `src/032_cosx_importance.c`

Analysis script   `src/032_cox_importance.R`

Executable        `exe/032_cosx_importance`

This exercise asked to compute the integral

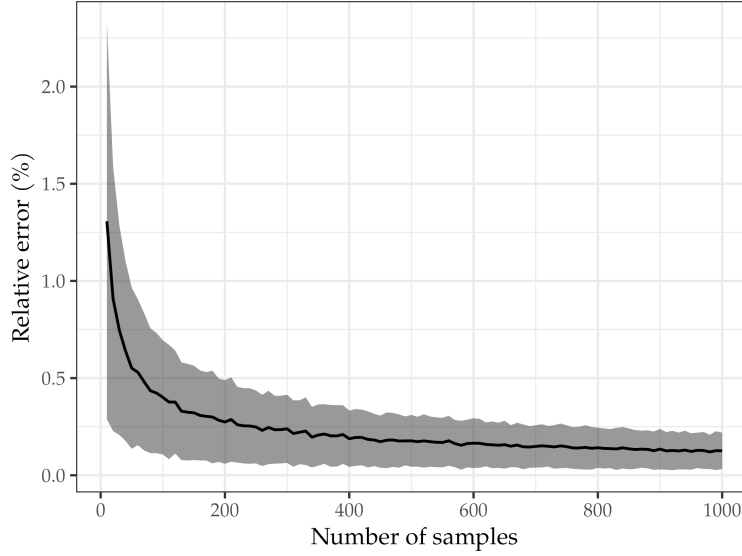
$$\int_0^{\pi/2} \cos x \, dx \quad (3.2.1)$$

using once again the importance sampling technique, with a weighting function  $g(x) = a + bx^2$ ,  $a$  and  $b$  to tune. To have the best performance, the idea is to have a  $g(x)$  that is large where the integrand is large and small when the integrand is small. First,  $g(x)$  should be normalized, which brings down the number of parameters to one:

$$1 = \int_0^{\pi/2} (a + bx^2) dx = \frac{\pi}{2}a + \frac{\pi^3}{24}b \implies a = \frac{2}{\pi} - \frac{\pi^2}{12}b. \quad (3.2.2)$$

Next, since we want  $g(x)$  to be as similar as possible to  $\cos x$ , we can restrict ourselves to  $b < 0$ . If  $b$  is too negative, though,  $g(x)$  becomes negative in part of the domain. The best solution is to impose  $g(\pi/2) = 0$ :

$$0 = \frac{2}{\pi} - \frac{\pi^2}{12}b + \frac{\pi^2}{4}b \implies b = -\frac{12}{\pi^3} \implies g(x) = \frac{3}{\pi} \left( 1 - \frac{4}{\pi^2}x^2 \right). \quad (3.2.3)$$



**Figure 3.2.1:** relative error in the integration of  $\cos x$  between 0 and  $\pi/2$  with the importance sampling technique, as described in exercise 3.2. The solid lines represent the average over 500 realizations, with the standard deviation shaded in grey.

To sample from  $g$ , we compute the cumulative as usual:

$$G(x) = \frac{3}{\pi} \int_0^x \left(1 - \frac{4}{\pi^2} y^2\right) dy = \frac{3}{\pi} x - \frac{4}{\pi^3} x^3. \quad (3.2.4)$$

The inversion requires solving a cubic equation,

$$u = \frac{3}{\pi} x - \frac{4}{\pi^3} x^3 \implies x^3 - \frac{3}{4} \pi^2 x + \frac{\pi^3}{4} u = 0. \quad (3.2.5)$$

Let's define  $\alpha = \pi^2/4$ :

$$x^3 - 3\alpha x + \pi\alpha u = 0. \quad (3.2.6)$$

This is a depressed cubic, so thankfully it has a trigonometric solution given by<sup>1</sup>

$$x_k = 2\sqrt{\alpha} \cos \left[ \frac{1}{3} \arccos \left( -\frac{3\pi\alpha u}{6\alpha} \sqrt{\frac{\beta}{\beta\alpha}} \right) - \frac{2\pi}{3} k \right], \quad \text{with } k = 0, 1, 2. \quad (3.2.7)$$

With some algebra, and selecting the right  $k$ , we can arrive at the much simpler form

$$x = \pi \sin \left( \frac{1}{3} \arcsin u \right), \quad \text{with } u \sim \mathcal{U}_{[0,1]}. \quad (3.2.8)$$

To wrap up, what I ended up doing is

$$\int_0^{\pi/2} \cos x \, dx \approx \frac{1}{N} \sum_{i=1}^N \frac{\cos x_i}{g(x_i)} = \frac{\pi}{3N} \sum_{i=1}^N \frac{\cos x_i}{1 - 4x_i^2/\pi^2}, \quad (3.2.9)$$

with each  $x_i$  generated according to eq. (3.2.8). The integration error as a function of the number of iterations is reported in fig. 3.2.1. Remarkably as low as 100 or 200 iterations are sufficient to get, on average, an error that is comfortably under 1%.

1. [https://en.wikipedia.org/wiki/Cubic\\_equation#Trigonometric\\_and\\_hyperbolic\\_solutions](https://en.wikipedia.org/wiki/Cubic_equation#Trigonometric_and_hyperbolic_solutions).

### 3.3 Analysing the variance with or without importance sampling

*Analysis script*    `src/033_variance_comparison.R`

This exercise asked to investigate the improvement one can get with importance sampling applied to a simple integral, the average on the distribution  $\rho(x) = e^{-x}$ ,  $x \geq 0$ , of the function

$$f(x) = \begin{cases} 0 & \text{for } 0 \leq x < T, \\ 1 & \text{for } x \geq T. \end{cases} \quad (3.3.1)$$

This average can be readily calculated as

$$\langle f \rangle_\rho = \int_0^\infty f(x) e^{-x} dx = \int_T^\infty e^{-x} dx = e^{-T}. \quad (3.3.2)$$

The problem of this integral is that  $f(x)$  is different from 0 in the region where  $\rho(x)$  is quickly vanishing. This means that if we were to sample numbers from  $\rho(x)$  we would waste a lot of iterations.

Consider instead the function  $g(x; a) = ae^{-ax}$  defined for  $x \geq 0$  and  $a \in (0, 1]$ . We can use it for an importance sampling and optimize  $a$  to minimize the variance. In particular, defining  $F(x) := f(x)\rho(x)/g(x)$  we can write

$$\langle f \rangle_\rho = \int_0^\infty f(x)\rho(x) dx = \int_0^\infty \frac{f(x)\rho(x)}{g(x)} g(x) dx = \langle F \rangle_g. \quad (3.3.3)$$

Let's calculate the second moment now:

$$\begin{aligned} \langle F^2 \rangle_g &= \int_0^\infty \frac{f^2(x)\rho^2(x)}{g^2(x)} g(x) dx = \int_T^\infty \frac{\rho^2(x)}{g(x)} dx \\ &= \frac{1}{a} \int_T^\infty e^{-(2-a)x} dx = \frac{e^{-(2-a)T}}{a(2-a)}. \end{aligned} \quad (3.3.4)$$

By subtracting the first moment squared we get the variance,

$$\sigma_g^2(F) = \langle F^2 \rangle_g - \langle F \rangle_g^2 = \frac{e^{-(2-a)T}}{a(2-a)} - e^{-2T}. \quad (3.3.5)$$

The variance of  $f$  with respect to  $\rho$  is given instead by

$$\begin{aligned} \sigma_\rho^2(f) &= \int_0^\infty (f(x) - \langle f \rangle_\rho)^2 \rho(x) dx = \int_T^\infty \rho(x) dx + \langle f \rangle_\rho^2 - 2\langle f \rangle_\rho \int_T^\infty \rho(x) dx \\ &= \langle f \rangle_\rho + \langle f \rangle_\rho^2 - \langle f \rangle_\rho^2 \\ &= e^{-T}(1 - e^{-T}). \end{aligned} \quad (3.3.6)$$

Now, let's minimize  $\sigma_g^2(F)$  with respect to  $a$ :

$$\frac{\partial}{\partial a} \sigma_g^2(F) = e^{-(2-a)T} \left[ \frac{Ta(2-a) - 2(1-a)}{a^2(2-a)^2} \right], \quad (3.3.7)$$

which gets us to

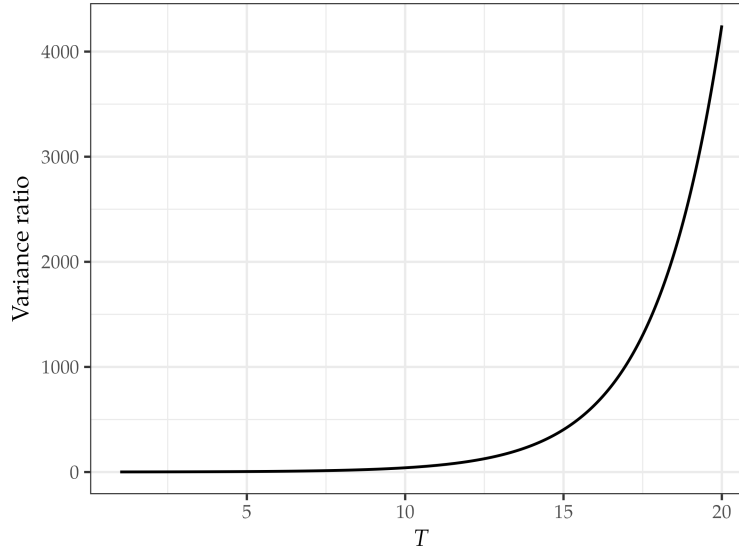
$$Ta^2 - 2(1+T)a + 2 = 0 \iff a = a_\pm^* = \frac{1+T \pm \sqrt{1+T^2}}{T}. \quad (3.3.8)$$

Only  $a_-^*$  is compatible with the condition  $a \in (0, 1]$ , so the optimal  $a$  is given by

$$a^* = \frac{1+T - \sqrt{1+T^2}}{T}. \quad (3.3.9)$$

**Table 3.3.1:** variance ratios for some values of  $T$ , all evaluated at  $a^*$ .

| $T$ | $\sigma_\rho(f)/\langle f \rangle_\rho$ | $\sigma_g(F)/\langle F \rangle_g$ | $\sigma_\rho(f)/\sigma_g(F)$ |
|-----|-----------------------------------------|-----------------------------------|------------------------------|
| 3   | 4.37                                    | 1.95                              | 2.24                         |
| 5   | 12.14                                   | 2.55                              | 4.76                         |
| 10  | 148.41                                  | 3.65                              | 40.71                        |
| 20  | 22026.47                                | 5.18                              | 4249.17                      |

**Figure 3.3.1:** variance ratio  $\frac{\sigma_\rho(f)}{\sigma_g(F)}$  as a function of  $T$ , from exercise 3.3.

In table 3.3.1 I calculated the ratios between each of the two variances and  $\langle f \rangle_\rho$  and the ratio between the two variances, for some values of  $T$ . While  $\sigma_g(F)/\langle f \rangle_\rho$  grows slowly,  $\sigma_\rho(f)/\langle f \rangle_\rho$  has an exponential increase; indeed

$$\frac{\sigma_\rho(f)}{\langle f \rangle_\rho} = \frac{\sqrt{e^{-T}(1 - e^{-T})}}{e^{-T}} = \sqrt{e^T - 1}. \quad (3.3.10)$$

The variance ratio as function of  $T$  is also plotted in fig. 3.3.1.



## Markov chains

### 4.1 Rewriting the recurrence relation

If we define the row vector of state probabilities at time  $n$  as  $\mu_n$  and the stochastic matrix as  $\mathcal{P}$ , we can write

$$\mu_n = \mu_{n-1} \mathcal{P}. \quad (4.1.1)$$

The  $(i, j)$ -th entry of  $\mathcal{P}$ ,  $p_{ij}$ , is the probability to go from state  $i$  to state  $j$ . If we work out the vector-matrix product in the previous formula, we get, for state  $i$ ,

$$\mu_n(i) = \sum_{j \in S} p_{ji} \mu_{n-1}(j), \quad (4.1.2)$$

where  $S$  denotes the set of possible states. Then, we can split the sum and obtain

$$\mu_n(i) = p_{ii} \mu_{n-1}(i) + \sum_{j \neq i} p_{ji} \mu_{n-1}(j). \quad (4.1.3)$$

The first term is representative of the situation in which the system was in the same state  $i$  at time  $n - 1$ , while the second represents all the possible transitions from another state to  $i$ . We can also rewrite the first term using the fact that the rows of the stochastic matrix are normalized:

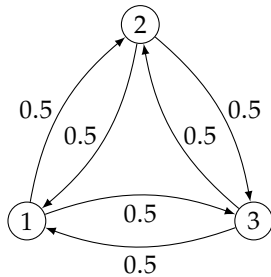
$$\sum_{j \in S} p_{ij} = 1 \implies p_{ii} = 1 - \sum_{j \neq i} p_{ij}. \quad (4.1.4)$$

In this way we obtain the formula we needed to prove,

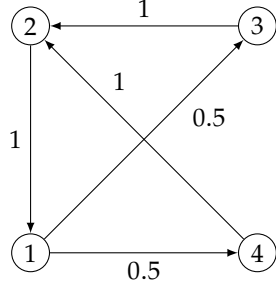
$$\mu_n(i) = \left(1 - \sum_{j \neq i} p_{ij}\right) \mu_{n-1}(i) + \sum_{j \neq i} p_{ji} \mu_{n-1}(j). \quad (4.1.5)$$

### 4.2 Drawing digraphs

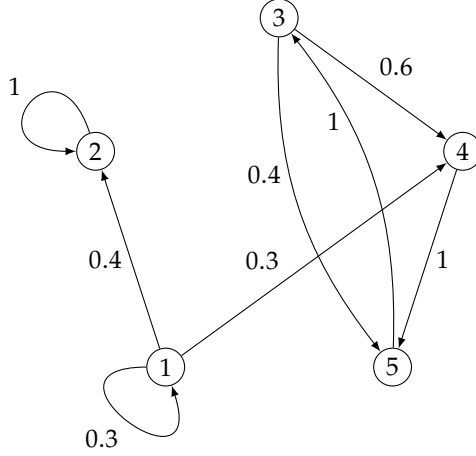
Below is the set of directed graphs corresponding to the given stochastic matrices.



$$\mathcal{P} = \begin{pmatrix} 0 & 0.5 & 0.5 \\ 0.5 & 0 & 0.5 \\ 0.5 & 0.5 & 0 \end{pmatrix} \quad (4.2.1)$$



$$\mathcal{P} = \begin{pmatrix} 0 & 0 & 0.5 & 0.5 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \quad (4.2.2)$$



$$\mathcal{P} = \begin{pmatrix} 0.3 & 0.4 & 0 & 0 & 0.3 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.6 & 0.4 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix} \quad (4.2.3)$$

### 4.3 Proving the irreducibility of a chain

Let's consider the Markov chain defined by the matrix

$$\mathcal{P}_1 = \begin{pmatrix} 1/2 & 1/2 \\ 1 & 0 \end{pmatrix}. \quad (4.3.1)$$

Is it irreducible? For every pair of states but one,  $(2, 2)$ , the corresponding entry in  $\mathcal{P}_1$  is different from zero. So, we can directly reach 1 from 1 and 2, and 2 from 1. We have to find a path of any length  $n$  that starts from 2 and comes back to 2, i.e. a  $n$  such that  $p_{ij}^n > 0$ . Let's see:

$$\mathcal{P}_1^2 = \begin{pmatrix} 3/4 & 1/2 \\ 1/2 & 1/2 \end{pmatrix}. \quad (4.3.2)$$

So, there is a path of length 2 that starts from 2 and ends back there – indeed, we can go from 2 to 1 with probability 1 and then from 1 to 2 with probability 1/2. Therefore, the Markov chain described by  $\mathcal{P}_1$  is irreducible.

To find  $\mathcal{P}_1^n$ , it is best to diagonalize the matrix first:

$$\mathcal{P}_1^n = T \mathcal{D}^n T^{-1}. \quad (4.3.3)$$

The transformation matrix is given by

$$T = \begin{pmatrix} 1 & 1 \\ 1 & -2 \end{pmatrix}, \quad (4.3.4)$$

while the diagonal matrix to the  $n$ -th power is

$$\mathcal{D}^n = \begin{pmatrix} 1 & 0 \\ 0 & -1/2 \end{pmatrix}^n = \begin{pmatrix} 1 & 0 \\ 0 & (-2)^{-n} \end{pmatrix}. \quad (4.3.5)$$

Putting all together we obtain

$$\mathcal{P}_1^n = \begin{pmatrix} 1 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 2^{-n} \end{pmatrix} \begin{pmatrix} 2/3 & 1/3 \\ 1/3 & -1/3 \end{pmatrix} = \frac{1}{3} \begin{pmatrix} 2 + (-1)^n 2^{-n} & 1 - (-1)^n 2^{-n} \\ 2 - (-1)^n 2^{1-n} & 1 + (-1)^n 2^{1-n} \end{pmatrix}. \quad (4.3.6)$$

In the limit  $n \rightarrow \infty$  this becomes

$$\lim_{n \rightarrow \infty} \mathcal{P}_1^n = \begin{pmatrix} 2/3 & 1/3 \\ 2/3 & 1/3 \end{pmatrix}. \quad (4.3.7)$$

Now, for the second matrix:

$$\mathcal{P}_2 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1/4 & 0 & 3/4 \end{pmatrix}, \quad (4.3.8)$$

let's find directly  $\mathcal{P}_2^n$ . The transformation matrix is

$$T = \begin{pmatrix} -4 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}, \quad (4.3.9)$$

while the diagonalized one to the  $n$ -th power is

$$\mathcal{D}^n = \begin{pmatrix} -1/4 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}^n = \begin{pmatrix} (-1)^n 4^{-n} & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (4.3.10)$$

Putting everything together,

$$\begin{aligned} \mathcal{P}_2^n &= \begin{pmatrix} -4 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} (-1)^n 4^{-n} & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1/5 & 0 & 1/5 \\ 1/5 & 0 & 4/5 \\ 0 & 1 & 0 \end{pmatrix} \\ &= \frac{1}{5} \begin{pmatrix} 1 + (-1)^n 4^{1-n} & 0 & 4 - (-1)^n 4^{1-n} \\ 0 & 5 & 0 \\ 1 - (-1)^n 4^{-n} & 0 & 4 + (-1)^n 4^{-n} \end{pmatrix}. \end{aligned} \quad (4.3.11)$$

Therefore, since  $\mathcal{P}_2^n$  has entries which are zero, the corresponding chain is *not* irreducible. The limit  $n \rightarrow \infty$  is

$$\lim_{n \rightarrow \infty} \mathcal{P}_2^n = \begin{pmatrix} 1/5 & 0 & 4/5 \\ 0 & 1 & 0 \\ 1/5 & 0 & 4/5 \end{pmatrix}. \quad (4.3.12)$$

#### 4.4 Proving the regularity of a chain

The goal here is to prove whether the Markov chain defined by a given matrix is regular. To do it, it suffices to find a  $n$  such that every entry of  $\mathcal{P}^n$  is positive. This implies that every entry of  $\mathcal{P}^m$  is positive whenever  $m \geq n$ . In the first case it is simple,  $n = 2$  suffices to end the proof:

$$\begin{pmatrix} 1/2 & 1/4 & 1/4 \\ 1/2 & 0 & 1/2 \\ 1/4 & 1/4 & 1/2 \end{pmatrix}^2 = \frac{1}{16} \begin{pmatrix} 7 & 3 & 6 \\ 6 & 4 & 6 \\ 6 & 3 & 7 \end{pmatrix}. \quad (4.4.1)$$

The second matrix instead is problematic:

$$\begin{pmatrix} 1 & 0 \\ 1/2 & 1/2 \end{pmatrix}. \quad (4.4.2)$$

If we look at it, we can realize that we cannot reach state 2 from state 1 in any way. So, the associated chain is reducible and irregular.

#### 4.5 Proving the stationarity of a chain

In this exercise, the first stochastic matrix is

$$\mathcal{P} = \begin{pmatrix} p & 1-p & 0 & 0 \\ 0 & 0 & p & 1-p \\ p & 1-p & 0 & 0 \\ 0 & 0 & p & 1-p \end{pmatrix}. \quad (4.5.1)$$

Let's try to skip the calculations this time. If we consider state 1, from the first row we see that we can reach 2 directly; from 2 we can then reach 3 and 4. From 2 we can reach 3 and 4; to reach 1 we can pass through 3, since  $p_{31} \neq 0$ . 3 and 4 present the same situation as 1 and 2 respectively. We can thus conclude that the associated Markov chain is recurrent irreducible.

As for periodicity, if we square the matrix we get

$$\mathcal{P}^2 = \begin{pmatrix} p^2 & p(1-p) & p(1-p) & (1-p)^2 \\ p^2 & p(1-p) & p(1-p) & (1-p)^2 \\ p^2 & p(1-p) & p(1-p) & (1-p)^2 \\ p^2 & p(1-p) & p(1-p) & (1-p)^2 \end{pmatrix}, \quad (4.5.2)$$

and this stays the same for all subsequent powers:

$$\mathcal{P}^3 = \begin{pmatrix} p^2 & p(1-p) & p(1-p) & (1-p)^2 \\ p^2 & p(1-p) & p(1-p) & (1-p)^2 \\ p^2 & p(1-p) & p(1-p) & (1-p)^2 \\ p^2 & p(1-p) & p(1-p) & (1-p)^2 \end{pmatrix}, \quad (4.5.3)$$

and so on. So, the chain is definitely aperiodic, since all the entries of  $\mathcal{P}^n$  for  $n > 1$  are positive.

To find the fixed point, we have to solve for  $\pi$  the equation

$$\pi\mathcal{P} = \pi \iff \mathcal{P}^T \pi^T = \pi^T \iff (\mathcal{P}^T - \mathbb{I})\pi^T = 0. \quad (4.5.4)$$

This leads to the system of equations

$$\begin{cases} (p-1)\pi_1 + \pi_3 = 0, \\ (1-p)\pi_1 - \pi_2 + (1-p)\pi_3 = 0, \\ p\pi_2 - \pi_3 + \pi_4 = 0, \\ (1-p)\pi_2 - p\pi_4 = 0. \end{cases} \implies \dots \implies \begin{cases} \pi_1 = \frac{p^2}{(1-p)^2}\pi_4, \\ \pi_2 = \frac{p}{1-p}\pi_4, \\ \pi_3 = \frac{p}{1-p}\pi_4. \end{cases} \quad (4.5.5)$$

Therefore, given a  $q \in [0, 1]$ , the corresponding stationary  $\pi$  is

$$\pi_q = \left( \frac{p^2 q}{(1-p)^2}, \frac{pq}{1-p}, \frac{pq}{1-p}, q \right) \quad (4.5.6)$$

The second part of the exercise asked to find another stationary distribution. The matrix is

$$\mathcal{P} = \begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/4 & 1/2 & 1/4 \\ 0 & 1/2 & 1/2 \end{pmatrix}. \quad (4.5.7)$$

The system corresponding to the equation  $(\mathcal{P}^T - \mathbb{I})\pi^T = 0$  is then

$$\begin{pmatrix} -1/2 & 1/4 & 0 \\ 1/2 & -1/2 & 1/2 \\ 0 & 1/4 & -1/2 \end{pmatrix} \begin{pmatrix} \pi_1 \\ \pi_2 \\ \pi_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \implies \begin{cases} \pi_1 = \pi_3, \\ \pi_2 = 2\pi_3. \end{cases} \quad (4.5.8)$$

The stationary distribution is thus given by the state vectors

$$\pi_q = (q, 2q, q) \quad \text{with } q \in [0, 1]. \quad (4.5.9)$$

## Ising model with the Metropolis algorithm

### 5.1 Simulations at different temperatures

*Source code*        `src/051_ising_metropolis.c`

*Analysis script*   `src/051_ising_metropolis.R`

*Executable*        `exe/051_ising_metropolis`

For this and the following exercise, I implemented the Metropolis algorithm with Glauber dynamics and sequential updating to simulate the two-dimensional Ising model. The program outputs the total energy and magnetization after each sweep of  $N = L^2$  local spin flips.

In order to estimate correctly the observables of interest, after discarding an appropriate amount of equilibration time steps, I estimated the autocorrelation time



## Advanced simulations of the Ising model

### 6.1 Wolff cluster algorithm

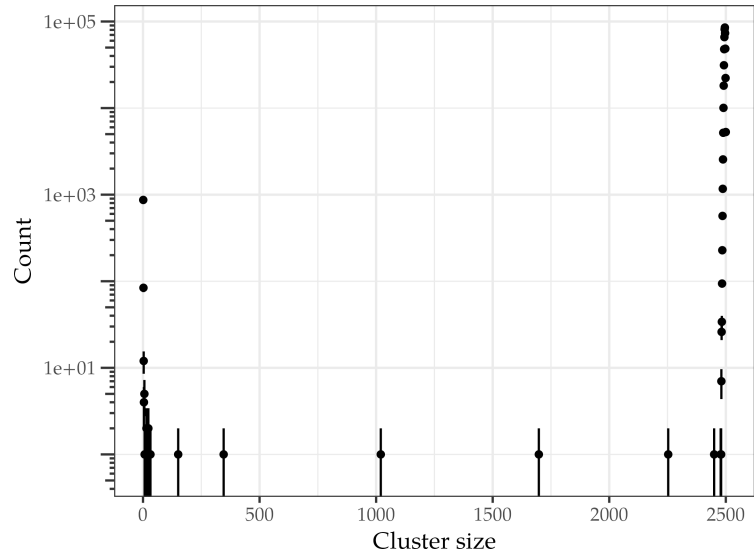
|                         |                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------|
| <i>Source codes</i>     | src/061_ising_wolff.c                                                               |
| <i>Analysis scripts</i> | src/061a_wolff_clusters.R src/061b_autocorrelation.sh<br>src/061b_autocorrelation.R |
| <i>Executable</i>       | exe/061_ising_wolff                                                                 |

For the first part of the exercise, I ran three simulations on a lattice of  $N = 50 \times 50$  spins at three different temperatures:  $T_c/2$ ,  $T_c$  and  $2T_c$ , each lasting 500 000 time steps. The histograms of cluster sizes, shown in figs. 6.1.1 to 6.1.3, exhibit expected profiles. The histogram at low temperature (fig. 6.1.1), is strongly bimodal with the largest peak around 2500 – corresponding to events where the entire lattice is flipped – and a rather smaller peak around 1, representing the clusters that do not spread beyond the first few spins. This is the expected behaviour in the ferromagnetic phase, since  $P_{\text{add}}$  is high. In contrast, the histogram at high temperature (fig. 6.1.3) shows an almost power-law distribution, with smaller sized clusters being predominant. The behaviour at the critical point (fig. 6.1.2) is intermediate between the ferromagnetic and paramagnetic histograms: the distribution is still bimodal, but cluster sizes near 1 are the most frequent, and the peak at larger sizes is wider and shifted to the left. Indeed, at the critical point we expect large ‘seas’ of correlated spins, though not encompassing the whole lattice.

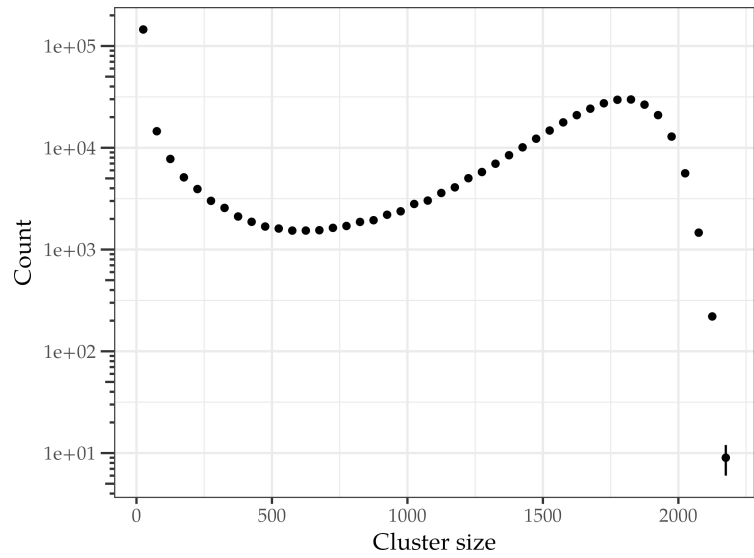
In the second part of the exercise, I examined the relationship between the autocorrelation time of the observed energy and magnetization with the lattice size,  $L$ , for both the Metropolis single spin-flip algorithm and the Wolff algorithm. The aim was to validate the scaling relation  $\tau \sim L^z$ , with the expectation that  $z$  would be smaller in the case of the Wolff algorithm. I conducted two sets of simulations, running each algorithm with  $L \in \{10, 14, 20, 28, 40, 50\}$  at the critical temperature  $T_c$  for 1 000 000 time steps. From each dataset, I computed the autocorrelation time of the energy and magnetization time series, after discounting the equilibration phase. For the Wolff algorithm, I adjusted  $\tau$  by multiplying it by a factor of  $S/L^2$ , with  $S$  being the average cluster size at the given temperature and lattice size.

A log-log plot of  $\tau$  as a function of  $L$ , separated by algorithm and observable, is presented in fig. 6.1.4. The parameters of the fitted lines are listed in table 6.1.1. The slope ( $z$ ) for Wolff’s energy is in good agreement with the expected value of  $z \approx 0.25$ , while the magnetization  $z$  is approximately half that. Another result in line with expectations is the smaller exponent  $z$  for the Wolff algorithm, observed for both energy and magnetization. In general, all autocorrelation times for the Metropolis algorithm are larger than those for the Wolff algorithm, particularly at larger lattice sizes. This is consistent with the expectation that the Wolff algorithm is significantly more efficient than the Metropolis algorithm near  $T_c$ .

### 6.2 Multiple Markov chains



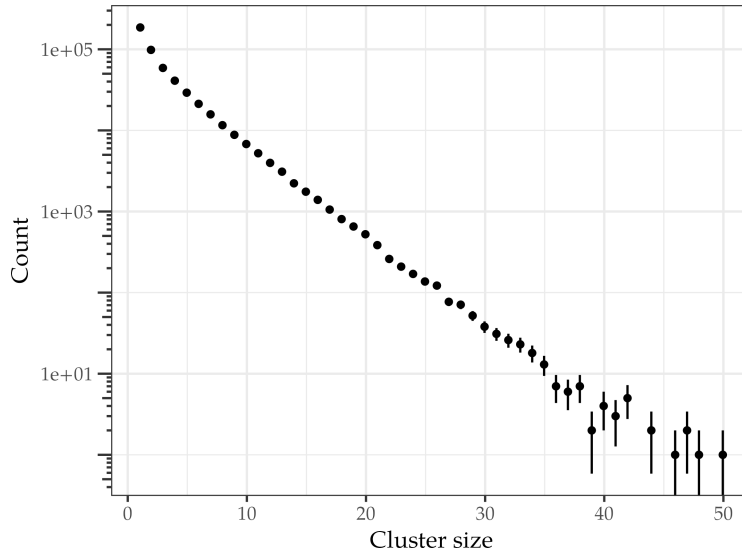
**Figure 6.1.1:** histogram of cluster sizes obtained from a Wolff algorithm simulation with  $N = 2500$  spins, run at a temperature  $T = T_c/2$  over 500 000 time steps. Error bars represent Poissonian uncertainties.



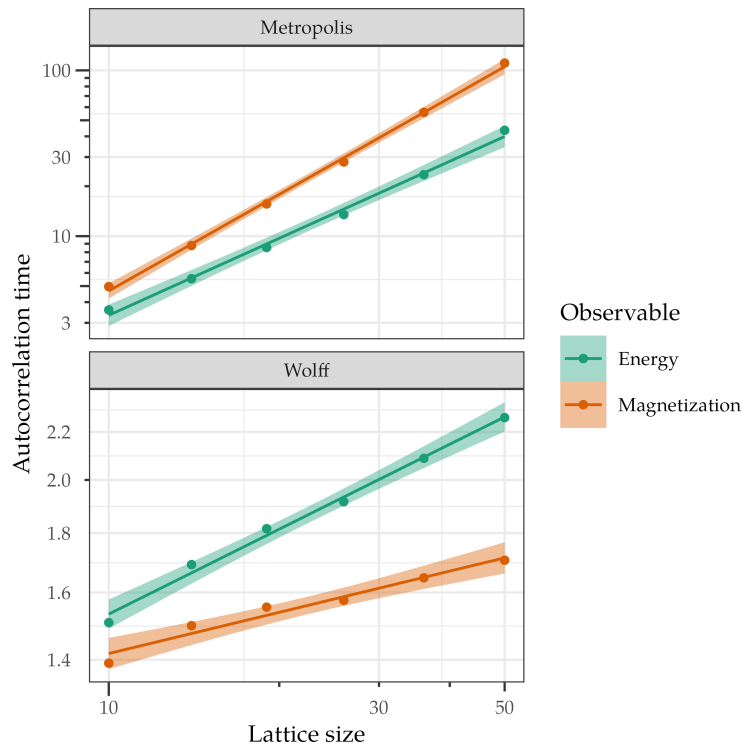
**Figure 6.1.2:** histogram of cluster sizes obtained from a Wolff algorithm simulation with  $N = 2500$  spins, run at a temperature  $T = T_c$  over 500 000 time steps. Error bars represent Poissonian uncertainties.

**Table 6.1.1:** parameters of the  $\log \tau \sim \log L$  linear fits from fig. 6.1.4.

| Algorithm  | Observable    | Parameter | Estimate | $t$ -statistic | $p$ -value           |
|------------|---------------|-----------|----------|----------------|----------------------|
| Metropolis | Energy        | Intercept | -2.35(2) | -14            | $1.7 \times 10^{-4}$ |
| Metropolis | Energy        | Slope     | 1.54(6)  | 28             | $9.6 \times 10^{-6}$ |
| Metropolis | Magnetization | Intercept | -2.9(1)  | -24            | $1.9 \times 10^{-5}$ |
| Metropolis | Magnetization | Slope     | 1.94(4)  | 49             | $1.0 \times 10^{-6}$ |
| Wolff      | Energy        | Intercept | -0.13(3) | -3.9           | $1.7 \times 10^{-2}$ |
| Wolff      | Energy        | Slope     | 0.24(1)  | 23             | $2.3 \times 10^{-5}$ |
| Wolff      | Magnetization | Intercept | 0.08(4)  | 2.1            | $9.9 \times 10^{-2}$ |
| Wolff      | Magnetization | Slope     | 0.12(1)  | 10             | $5.1 \times 10^{-4}$ |



**Figure 6.1.3:** histogram of cluster sizes obtained from a Wolff algorithm simulation with  $N = 2500$  spins, run at a temperature  $T = 2T_c$  over 500 000 time steps. Error bars represent Poissonian uncertainties.



**Figure 6.1.4:** autocorrelation time of energy and magnetization as a function of the lattice size for the Metropolis and Wolff algorithms. Each run lasted 1 000 000 time steps. The parameters of the fitted lines are reported in table 6.1.1.



## Continuous-time Markov processes

### 7.1 Lotka–Volterra model

*Source codes*      `src/071_lotka_volterra.c`, `src/ctmp/gillespie.h`,  
                          `src/ctmp/gillespie.c`, `src/ctmp/lotka_volterra.h`

*Analysis script*   `src/071_lotka_volterra.R`

*Executables*        `exe/071_lotka_volterra`

For this exercise and the next, I wrote a generic implementation of the Gillespie algorithm, found in `src/ctmp/gillespie.c`. The header file `src/ctmp/lotka_volterra.h` implements the specific rate and update functions for the Lotka–Volterra model. Then I ran a set of simulations fixing the three parameters to  $k_1 = 3 \text{ s}^{-1}$  (prey birth rate),  $k_2 = 0.01 \text{ s}^{-1}$  (predation rate) and  $k_3 = 5 \text{ s}^{-1}$  (predators death rate); the results are in figs. 7.1.1 and 7.1.2.

The non-trivial stationary solution is  $(k_3/k_2, k_1/k_2)$ , which equals (500, 300) with this choice of parameters. When the initial condition is close to this equilibrium point, as shown in the right middle panel of fig. 7.1.1 or the left middle panel of fig. 7.1.2, the oscillations can be relatively small and almost sinusoidal, with the predators’ trend mirroring that of the prey but with a delay. However, when one or both initial populations are very far from the (500, 300) point, as in the left bottom panel of fig. 7.1.1 or the bottom right panel of fig. 7.1.2, the oscillations become more pronounced and take on a characteristic shape. In this case, in addition to the delay in the predators’ trend, a sharp peak in both populations is followed by a drop – first of the prey population, which quickly recovers, and then of the predators, which recover only when the prey population reaches a new maximum.

When food is scarce, predators may go extinct and prey grow exponentially with rate  $k_1$  – this happens in the two upper panels of both fig. 7.1.1 and fig. 7.1.2. Predator extinction can also be prevented by decreasing the predator death rate and/or increasing the predation rate, provided that both the prey birth rate and predator death rate remain sufficiently high – otherwise, over-predation would drive the prey to extinction. For example, fig. 7.1.3 shows a combination of parameters,  $k_1$  and  $k_2$  as before but  $k_3$  lowered to  $4 \text{ s}^{-1}$ , that easily leads to a ‘well-behaved’ solution. But if we further lower the predator death rate to  $2 \text{ s}^{-1}$ , the predators can easily grow too quickly, leading to the extinction of both species, as shown in fig. 7.1.4.

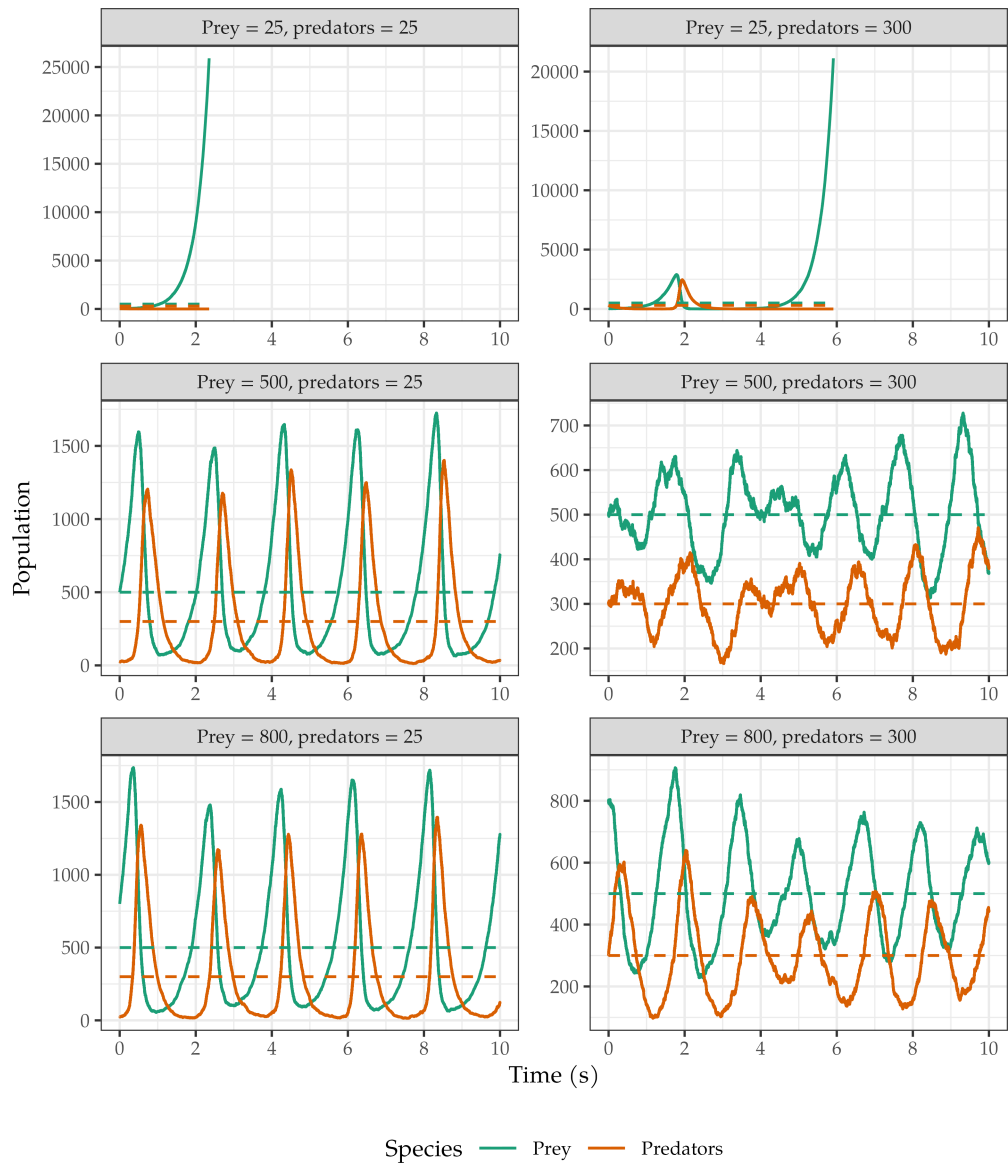
### 7.2 Brusselator model

*Source codes*      `src/072_brusselator.c`, `src/ctmp/gillespie.c`,  
                          `src/ctmp/gillespie.h`, `src/ctmp/brusselator.h`

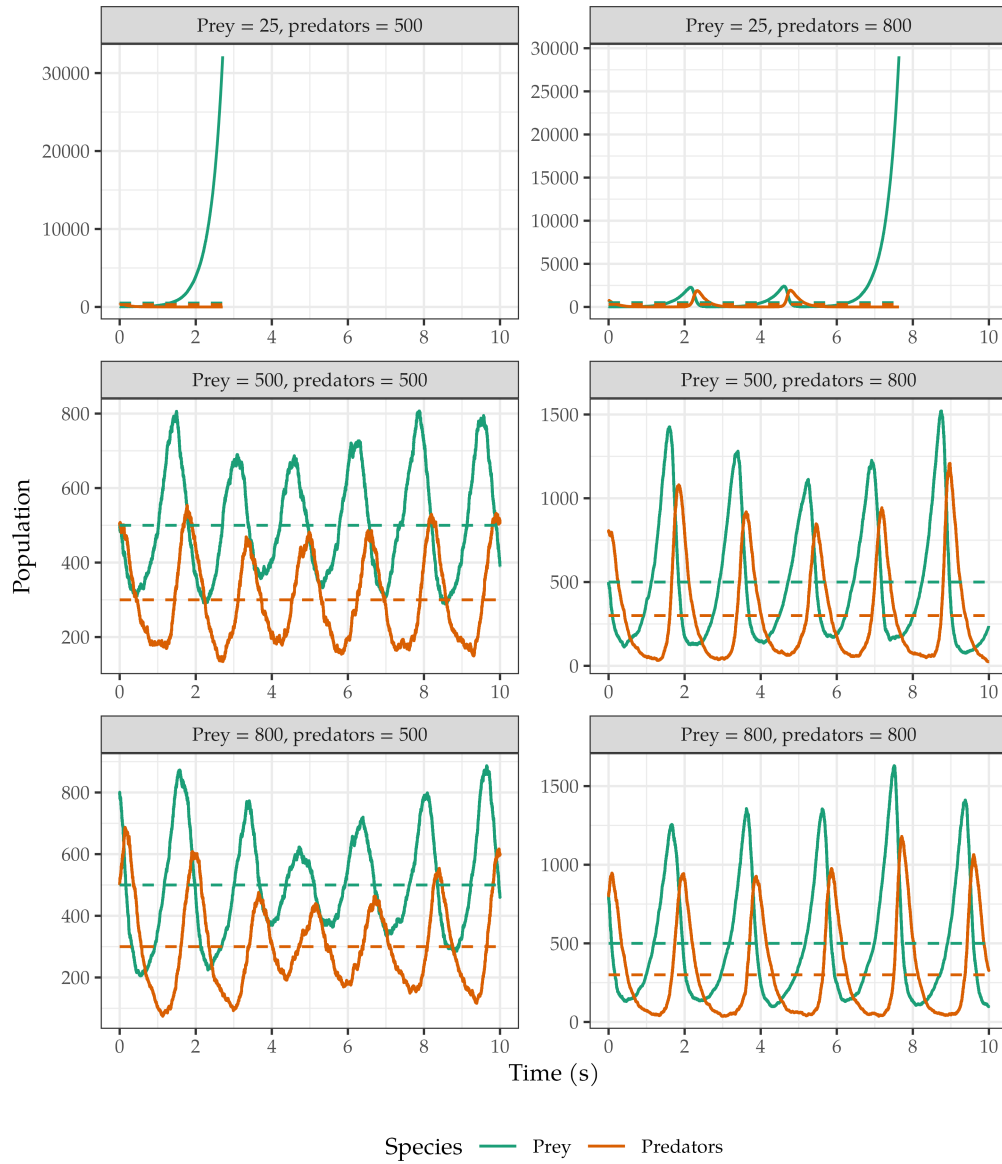
*Analysis script*   `src/072_brusselator.R`

*Executables*        `exe/072_brusselator`

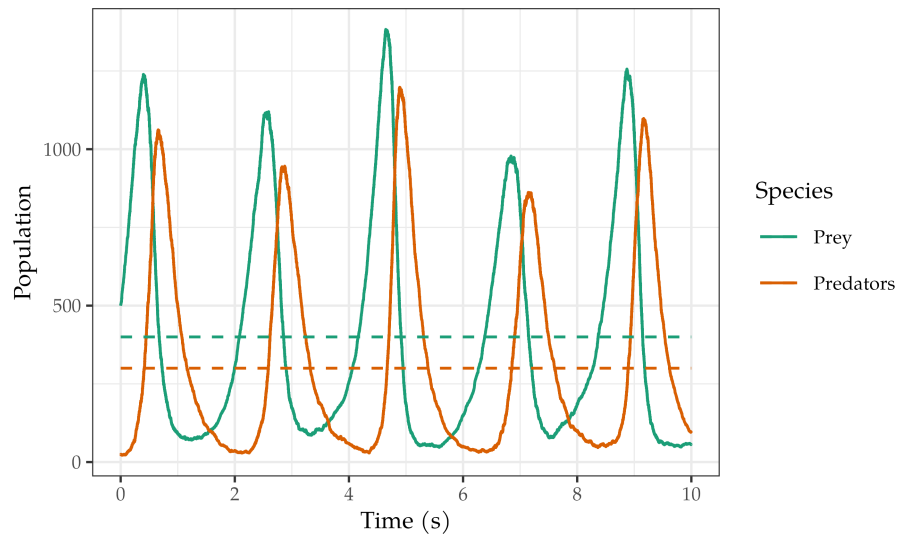
This exercise uses the same base implementation of the Gillespie algorithm found in `src/ctmp`, with a different header file that implements the rate and update functions of the



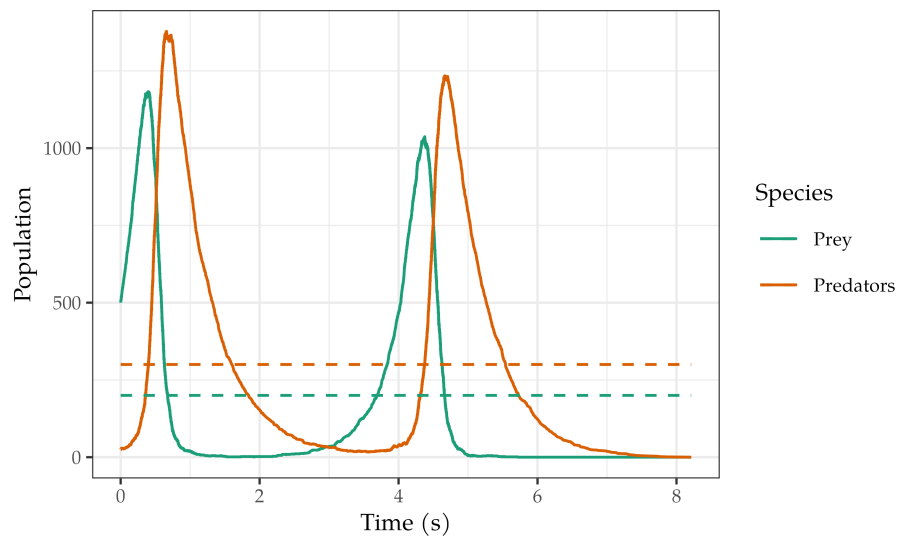
**Figure 7.1.1:** first set of Lotka–Volterra simulations with  $k_1 = 3 \text{ s}^{-1}$ ,  $k_2 = 0.01 \text{ s}^{-1}$  and  $k_3 = 5 \text{ s}^{-1}$ . The initial conditions are reported above each plot and the stationary solution  $(k_3/k_2, k_1/k_2)$  is highlighted with dashed lines.



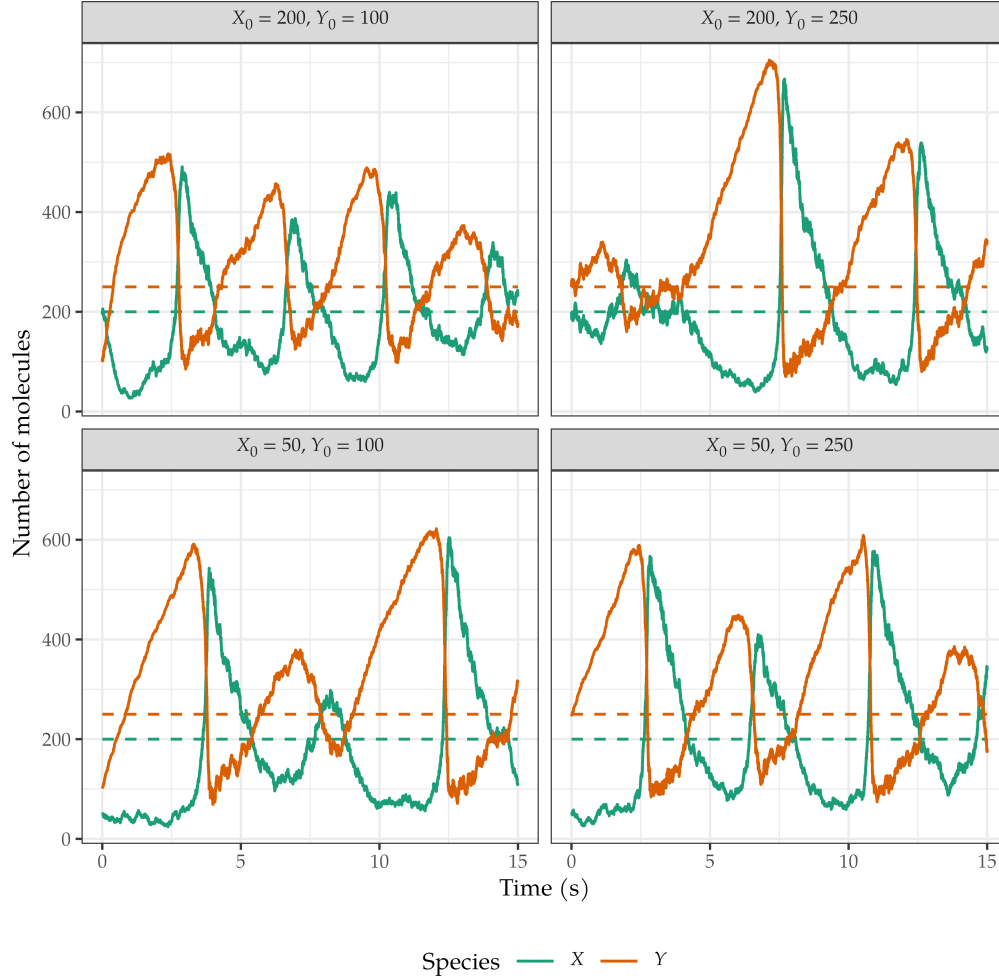
**Figure 7.1.2:** second set of Lotka–Volterra simulations with  $k_1 = 3 \text{ s}^{-1}$ ,  $k_2 = 0.01 \text{ s}^{-1}$  and  $k_3 = 5 \text{ s}^{-1}$ . The initial conditions are reported above each plot and the stationary solution ( $k_3/k_2, k_1/k_2$ ) is highlighted with dashed lines.



**Figure 7.1.3:** simulation of the Lotka–Volterra model with  $k_1 = 3 \text{ s}^{-1}$ ,  $k_2 = 0.01 \text{ s}^{-1}$  and  $k_3 = 4 \text{ s}^{-1}$ .



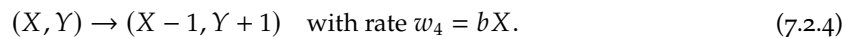
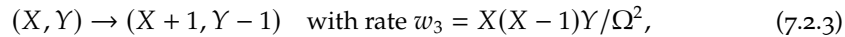
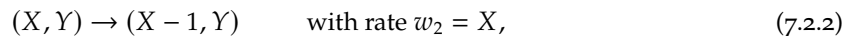
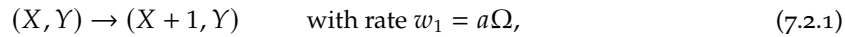
**Figure 7.1.4:** simulation of the Lotka–Volterra model with  $k_1 = 3 \text{ s}^{-1}$ ,  $k_2 = 0.01 \text{ s}^{-1}$  and  $k_3 = 2 \text{ s}^{-1}$ .



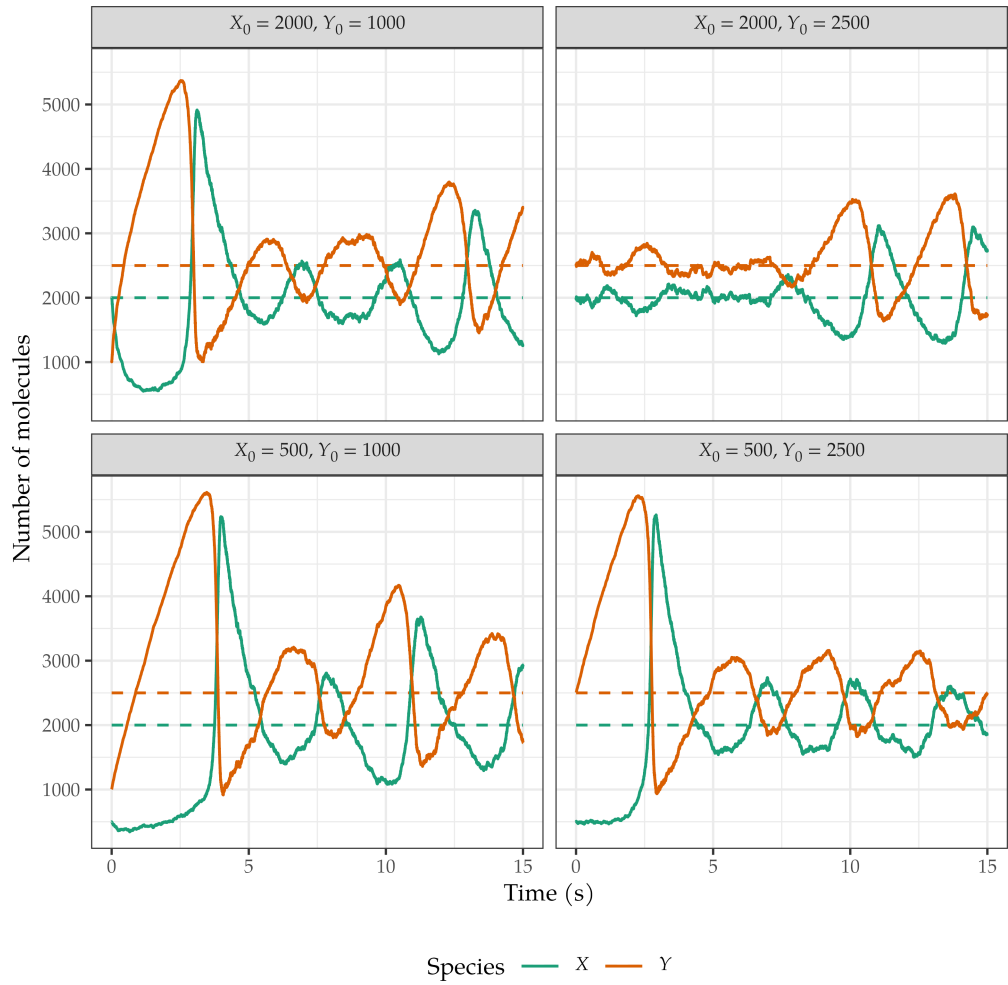
**Figure 7.2.1:** first set of Brusselator simulations, with  $a = 2 \text{ s}^{-1}$ ,  $b = 5 \text{ s}^{-1}$  and volume  $\Omega = 10^2$ . The initial conditions are reported above each plot and the stationary solution  $(a\Omega, b\Omega/a)$  is highlighted with dashed lines.

Brusselator model, located at `src/ctmp/brusselator.h`. I ran three set of simulations using fixed parameters  $a = 2 \text{ s}^{-1}$  and  $b = 5 \text{ s}^{-1}$  while varying the ‘volume’  $\Omega = \{10^2, 10^3, 10^4\}$ ; the results are presented in figs. 7.2.1 to 7.2.3. For each value of  $\Omega$ , I considered four initial conditions:  $(X_0, Y_0) = (2\Omega, \Omega)$ , shown in the upper left panel of each figure,  $(2\Omega, 5\Omega/2)$  in the upper right panel,  $(\Omega/2, \Omega)$  in the lower left panel and  $(\Omega/2, 5\Omega/2)$  in the lower right panel. These choices intend to explore the vicinity of the stationary solution  $(a\Omega, a\Omega/b) = (2\Omega, 5\Omega/2)$ .

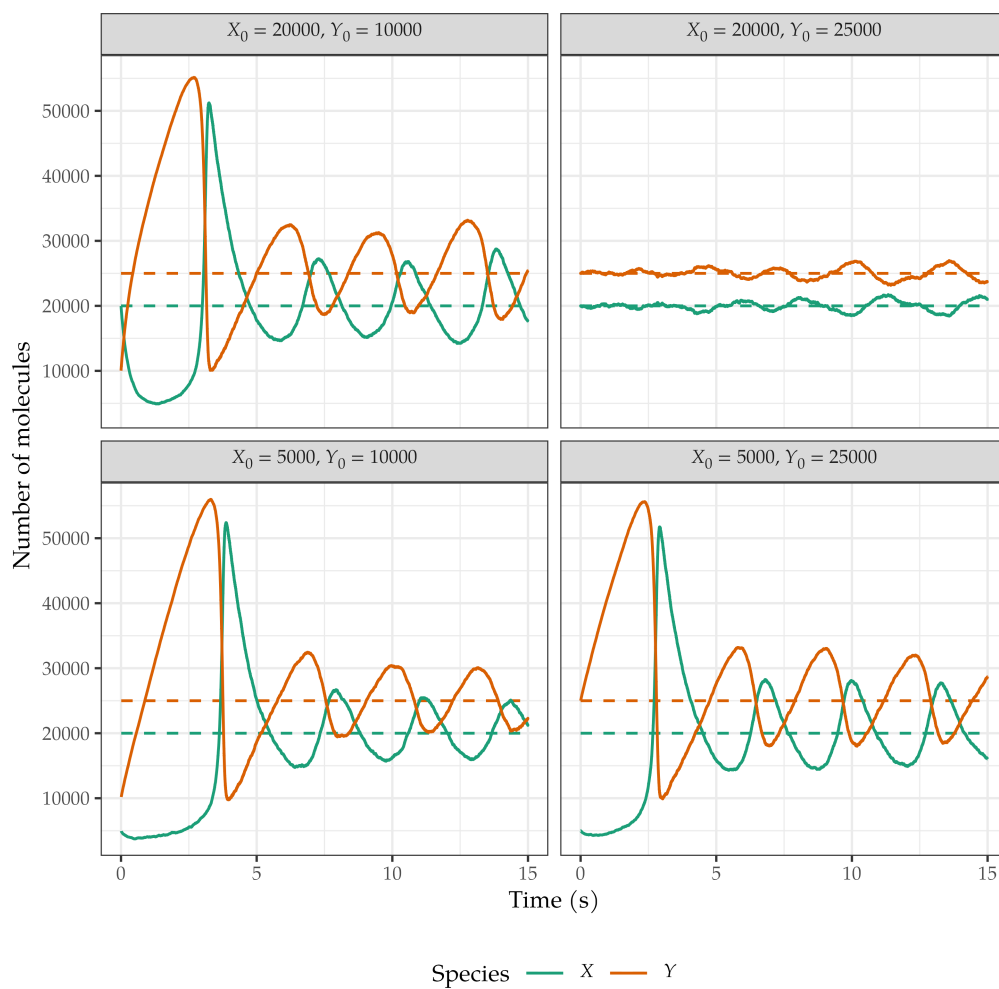
To understand what happens, let’s refer to the four transitions of the model:



With  $b > a$  the fourth reaction is favoured for every chosen initial condition. This is evident in the sharp rise in  $Y$  observed in all cases except the upper right panels, where both populations are large enough to favour the third reaction, which competes with the fourth. A recurring feature is a sudden decline in  $Y$  accompanied by a corresponding rise in  $X$ : this is again triggered by the third reaction, which becomes more frequent as  $Y$  grows – and less frequent at large volumes, explaining the generally less ‘spiky’ behaviour in fig. 7.2.3.



**Figure 7.2.2:** first set of Brusselator simulations, with  $a = 2 \text{ s}^{-1}$ ,  $b = 5 \text{ s}^{-1}$  and volume  $\Omega = 10^3$ . The initial conditions are reported above each plot and the stationary solution  $(a\Omega, b\Omega/a)$  is highlighted with dashed lines.



**Figure 7.2.3:** first set of Brusselator simulations, with  $a = 2 \text{ s}^{-1}$ ,  $b = 5 \text{ s}^{-1}$  and volume  $\Omega = 10^4$ . The initial conditions are reported above each plot and the stationary solution  $(a\Omega, b\Omega/a)$  is highlighted with dashed lines.





*Executable*      `exe/082_off_lattice_mc`

The four source files listed above form a flexible base for implementing an off-lattice Monte Carlo simulation. `parameters.c` and `parameters.h` define a struct `parameters` to hold the simulation parameters that are fed to the program through a configuration file with a `keyword_value` format. The function `parse_config` reads the configuration file and updates the struct `parameters` it receives. The parameters to set are:

`num_particles` number of particles.

`box_size` length of the (cubic) simulation box's side, in reduced units.

`density` in reduced units. Only two between this and the two previous two parameters need be specified; `parse_config` calculates the remaining one.

`temperature` in reduced units.

`max_disp` maximum displacement in each Cartesian direction of a Monte Carlo trial move, in reduced units.

`num_steps` number of Monte Carlo sweeps (each comprising `num_particles` trial moves).

`num_realizations` number of system realizations.

`init_conf` initialization strategy, either random or (cubic) lattice.

`integration.c` holds two functions that take the parameters structure updated through the configuration file to run the specified simulation. The first function, `initialize`, populates a double array of  $3 \times \text{num\_particles}$  elements (flattened) with particle coordinates determined by the strategy set with the `init_conf` parameter. The second function, `sweep`, performs a single Monte Carlo sweep of `num_particles` trial moves using the Metropolis algorithm. The sweep function requires a pointer to a `compute_energy_delta` function, specific to the system being simulated, which returns the energy difference resulting from the trial move. The header file `integration.h` also defines a struct `observables`, intended to store the current system energy and the acceptance ratio from the last sweep. The sweep function takes a struct `observables` as an argument and updates it accordingly.

### 8.3 Off-lattice Monte Carlo of hard spheres

*Source codes*      `src/083_hard_spheres.c`, `src/offlat/integration.c`,  
                          `src/offlat/integration.h`, `src/offlat/parameters.c`,  
                          `src/offlat/parameters.h`

*Analysis script*   `src/083_hard_spheres.R`

*Executable*        `exe/083_hard_spheres`